AI + IoT · Final week

Your final project.

Three weeks ago you were web developers. Now you have trained a model, wired a board and made the two talk over WiFi. Ten machines are on the next pages. Pick one, build its brain at home, and bring it to life in the lab.

Pick the one that keeps you up at night.

Software Innovation Center · AUCA · Yvan Nshuti and Rutayisire Kendra

READ THIS FIRST

How the final project works

Every project on the next pages is the finger counter's shape: a **brain** on your laptop (a model and your rules), a **cloud** (your backend, one truth, a few endpoints) and a **body** (the ESP32 and its parts). You already built one of these end to end. Now you build one that is yours.

1

Choose

Pick one project and tell us your choice in the form. Several students may pick the same project; the lab has the parts for it.

2

Build the brain and the cloud at home

The model, your rules, the backend and its endpoints. Test the whole chain on your laptop with a fake body: print what the board would do.

3

Write the sketch at home

Your ESP32 code, ready to upload: WiFi, the GET or POST to your backend, and the pins. The boards and parts stay in the lab, so the sketch is written blind and tested on the day.

4

Grading day: bring it to life

You come with your laptop, wire the board from our parts, upload your sketch, and show us the machine working. Then you explain each tier and how it fails.

The parts are shared, so not everyone is graded on the same day. Each student gets a slot. On your slot the board, the sensors, the servo and the LEDs are on the table for you. Everything else, the brain, the cloud and the sketch, arrives on your laptop, finished.

What we look for

✅ **It works end to end.** Camera or microphone in, decision in the cloud, motion or light out.

🧠 **You can explain each tier.** Any tier, in plain words. What the model sees, what the endpoint returns, what the pin does.

⚠️ **You know how it fails.** A photo instead of a face, a noisy room, a lost WiFi. Say it before we find it.

✨ **It is yours.** Your rules, your thresholds, your enrolment data, your wiring. The finger counter is your starting point, not your submission.

Before you choose

🎯 **One sense, one decision, one action.** Get the loop working first. Add a second trick only when the first one never fails.

📈 **Own your data.** Faces, claps, digits, alarm sounds: you record and enrol them yourselves. No dataset, no project.

💪 **Start from what works.** Your finger counter folder is the skeleton of every project here. Copy it, rename it, grow it.

Ten projects. Four families. One of them is yours.

FAMILY 1 · THE EYES

Vision

The laptop webcam and MediaPipe, exactly as in the finger counter. The camera looks, a model understands, your rule decides, the board acts.

1 🚪 Face plus card door

THE MISSION

Two factors, like a bank vault. Tap your RFID card on the reader, then show your face to the camera. The backend opens a ten second window for the card owner's face; if the face matches, the servo swings the door open, a green light and a short beep say welcome. Wrong face, no face, or too slow: red light, long buzz, door stays shut. You enrol your own face, and a friend's for the wrong face test.

YOU WILL USE

ESP32, the MFRC522 RFID reader and its cards, one servo, one red and one green LED, one buzzer, the laptop webcam.

The hardest on the list. Face recognition, not just detection: use OpenCV's LBPH recogniser and an enrolment step. A printed photo will fool it; know that, say that.

2 🚶 Lab people counter with a door

THE MISSION

A webcam on the door counts people as they arrive and leave. Each time a person is detected at the door, the servo opens for them and the OLED shows how many are inside. Above the limit you set, the door stays shut and the buzzer politely refuses until someone leaves.

YOU WILL USE

ESP32, one servo, the OLED screen, three LEDs for empty, busy and full, one buzzer, the laptop webcam.

The trick is counting a person once, not thirty times a second. A line the person crosses, and a short cool down, is enough.

3 🖐 Air writing

THE MISSION

Draw a digit in the air with one finger. The brain follows your fingertip, turns the path into a small image and reads it with a model you train yourselves (or Teachable Machine). The OLED shows the digit and four LEDs show it in binary. A wrong read is fine; a system that cannot say it is unsure is not.

YOU WILL USE

ESP32, the OLED screen, four LEDs, one push button to clear, the laptop webcam.

You collect your own digits to train on. Twenty drawings per digit is a fine dataset.

4 🖐 Hand tracker

THE MISSION

The board follows your hand. As your hand moves from the left edge of the picture to the right, the servo turns to point at it, and the LEDs follow: brightness by pinch, or count by fingers, your call. Smooth it so the servo does not shiver when your hand is still.

YOU WILL USE

ESP32, one servo, five LEDs, the OLED (optional), the laptop webcam.

The SG90 is a 180 degree servo: it turns half a circle and no further. Your mapping is the left edge to 0 and the right edge to 180, nothing outside that.

5 🅿 Camera parking guard

THE MISSION

One webcam above a toy parking lot made of tape and toy cars. The brain counts free spots and the backend knows the total. At the gate, green means drive in and the servo barrier lifts; red means full and the barrier stays down. The OLED shows the number of free spots, the way Kigali's malls do.

YOU WILL USE

ESP32, one servo, one red and one green LED, the OLED screen, the laptop webcam, tape and a few toy cars.

FAMILY 2 · THE EARS

Speech and Sound

The laptop microphone. Sound becomes words or labels on the laptop, offline, then it is a command in your cloud and an action on the board.

6 🎙️ Voice lights

THE MISSION

Say "lights on", "dim", "red", "party". Speech to text runs on the laptop (Vosk or Whisper, offline), your rules turn the sentence into a command, the backend stores it, and the board dims and switches the LEDs with PWM. The OLED shows the phrase it understood, so a wrong hearing is visible, not mysterious.

YOU WILL USE

ESP32, five LEDs, the OLED screen, the laptop microphone.

Plan for a noisy room: a push to talk key on the laptop beats an always on microphone on demo day.

7 👏 Clap lock

THE MISSION

Clap a rhythm you enrolled, the way you would knock on a friend's door, and the servo opens. A wrong rhythm buzzes and flashes red. The brain must tell a clap from a cough, a dropped pen or someone talking, and compare the timing between claps to the pattern you stored.

YOU WILL USE

ESP32, one servo, one buzzer, one red and one green LED, the laptop microphone.

8 🔔 Sound alarm forwarder

THE MISSION

The brain listens for a doorbell, breaking glass or a smoke alarm (train the sounds yourselves, phone recordings are fine). When it hears one, the board flashes and buzzes, and the backend sends you an email saying what it heard and when. A house that tells you about the alarm you could not hear.

YOU WILL USE

ESP32, three LEDs, one buzzer, the laptop microphone, an email account for the backend to send from.

The email is sent by the backend with Python's smtplib (Gmail with an app password works), never by the board. Secrets live in the cloud, not on a breadboard.

FAMILY 2 + 4 · EARS AND WORDS

Speech plus Language

A language model lives in your backend. Your words become its input, its answer is a JSON command the board can read.

9 💬 Talk to your room

THE MISSION

Speak to the room, or type to it when the room is loud: "I am studying, make it calm", "movie night", "wake me up". Speech to text gives the sentence, a language model in the backend turns the mood into a command with fields the board understands (brightness, colour, a servo angle), and the board obeys. The OLED shows what it understood and why.

YOU WILL USE

ESP32, five LEDs, the OLED screen, one servo (optional, a blind or a fan), the laptop microphone, a language model API key in the backend.

Ask the model to answer with JSON only, and validate it before the board ever sees it. The board never talks to the model; it reads what the cloud hands it.

FAMILY 4 · THE WORDS

Language

Plain English in, a rule out. The language model writes what a developer used to write, and your backend enforces it.

10  Rules in plain English

THE MISSION

Type a rule the way you would say it: "if the door opens after 9 pm, flash red", "buzz twice when the button is held for three seconds". The language model turns it into a structured rule, the backend stores it and checks every event against it, and the board acts. New rule, no redeploy, no reflash. The OLED shows the rule that fired.

YOU WILL USE

ESP32, one push button standing in for the door, three LEDs, one buzzer, the OLED screen, a language model API key in the backend.

Keep the rule format small: a trigger, a condition, an action. The model fills the format; your backend is the judge that runs it.

Tell us your project this week. Fill the form with your name, your ID and your project number before Tuesday 29 September, 12:00 PM. Then build the brain and the cloud, write the sketch, and come to your slot ready to wire, upload and show. *Bring something that moves.*